Week 7 — Memory, Kernel Tasks, and Scheduling

Format: seven daily sections collected into one mastery package

Suggested workload: approximately 21 focused hours, adjustable to mastery

> Keep one CPU mental image: memory decides what an address means; scheduling decides which saved instruction stream owns the CPU next.

This package studies the existing `projects/toyos` Week 7 milestone. The practical work invokes or inspects that canonical project; it does not fork the kernel. Every challenge follows:

```
purpose → prediction → evidence → explanation → limits
```

ToyOS schedules two **ring-0 kernel tasks**, not user processes. They share one address space and privilege level. Ring 3, syscalls, user ELF loading, per-process page tables, and process isolation are Week 8 topics.

Daily roadmap

Day 1 — Boot memory map and physical frames

Parse Multiboot2's variable-size memory-map tag and audit the deliberately bounded first-fit bitmap allocator.

- Lesson: day-001-memory-map-and-frame-allocator.md
- Challenge: `challenges/day-001-memory-frames.sh`
- Evidence: normal allocator marker plus deterministic double-free injection, checked marker/status, exact fault-profile symbols, and source invariants.

Day 2 — Page tables, TLBs, and page faults

Walk x86-64's four-level translation, ToyOS's mapper, `invlpg`, and the existing page-fault diagnosis; compare AArch64 and RISC-V mechanisms.

- Lesson: day-002-page-tables-mapper-and-faults.md
- Challenge: `challenges/day-002-mapper-fault.sh`
- Evidence: mapper source/disassembly and checked CR2/error-code output.

Day 3 — Kernel heap and ownership

Explain the bump heap's alignment and exhaustion checks, then make lifetime and ownership claims explicit.

- Lesson: day-003-kernel-heap-and-ownership.md
- Challenge: `challenges/day-003-kernel-heap.sh`
- Evidence: mapped heap construction, allocator checks, and task allocations.

Day 4 — Kernel tasks and saved contexts

Trace task records, dedicated stacks, trampoline entry, and saved contexts.

- Lesson: day-004-kernel-tasks-stacks-contexts.md
- Challenge: `challenges/day-004-task-context.sh`
- Evidence: C/assembly layout agreement, task-start markers, and deterministic corrupted-stack-alignment rejection before the first context switch.

Day 5 — Cooperative yield and ABI ownership

Follow a voluntary switch and account for callee-saved registers, stack ownership, return addresses, and scheduler state.

- Lesson: day-005-cooperative-yield-and-switch.md
- Challenge: `challenges/day-005-cooperative-switch.sh`
- Evidence: switch disassembly, ABI register set, and cooperative marker.

Day 6 — Timer preemption and interrupt safety

Follow PIT IRQ0 through the remapped PIC into scheduling, and contrast the legacy lab path with APIC/x2APIC systems.

- Lesson: day-006-pit-pic-preemption.md
- Challenge: `challenges/day-006-preemption.sh`
- Evidence: IRQ source path, tick growth, EOI ordering, and preemption marker.

Day 7 — Trace, diagnose, and compare

Build a scheduler report, diagnose injected allocator/page/context failures, and compare ToyOS with xv6 and Linux before Week 8.

- Lesson: day-007-scheduler-report-and-comparison.md
- Challenge: `challenges/day-007-scheduler-report.sh`
- Evidence: one temporary report with exact artifacts, tests, source contracts, all three injected fault profiles, interpretations, comparisons, and limits.

Integrated checkpoint

Narrate both chains without skipping ownership:

```
Multiboot map → free physical frame → page-table ownership → virtual mapping
→ heap object → task and stack → runnable → running → IRQ/yield
→ saved context → run queue choice → restored context
```

For every arrow, identify the architecture rule, ABI rule, ToyOS policy, mutable owner, and evidence. State the single-core/no-locking assumption.

Reference discipline

Use current architecture manuals for normative details. Linux source landmarks include `arch/x86/entry/`, `arch/x86/mm/`, `mm/`, and `kernel/sched/`; inspect and record a revision before making line-level claims. Current xv6-riscv concepts commonly live around `kernel/kalloc.c`, `kernel/vm.c`, `kernel/proc.c`, `kernel/swtch.S`, and `kernel/trap.c`, but verify the tree you cite. Local comparison material is available at `references/ARCHITECTURE_COMPARISON.md`.

Do not auto-fill mastery, confidence, hours, or completion.

Day 1 — Multiboot Memory Map and Bounded Physical Frames

Target time: approximately 3 hours

> RAM is not “everything below the installed-memory number.” Firmware and the bootloader describe ranges; the kernel must validate, clip, and reserve them.

1. Keep physical and virtual questions separate

A physical-frame allocator answers, “Which 4 KiB physical frame may the kernel own?” A mapper answers, “Which virtual address translates to which physical frame?” `malloc`-style code answers, “Which byte range may this caller use?” Mixing those questions causes excellent-looking but false debugging stories.

ToyOS begins with an identity-mapped first GiB, so a physical address can currently be cast to a pointer. That convenience is a bootstrap policy, not a general truth about paging.

2. Parse a tagged, variable-length handoff

GRUB passes the Multiboot2 information pointer into `kernel_main` in `projects/toyos/src/kernel.c`. In `projects/toyos/src/memory.c`, `memory_init` reads `total_size`, starts after the eight-byte fixed header, and advances each tag by its size rounded to eight bytes. For type 6 it also honors `entry_size`; hard-coding the local C struct size as the stride would reject compatible larger entries.

Before trusting a field, ask:

- Does the outer tag fit before `limit`?
- Is `tag->size` large enough for its fixed header?
- Is the memory-map entry stride large enough?
- Can `address + length` overflow?
- Is the range type 1, meaning available RAM?

ToyOS checks the central size and overflow conditions. A production parser would reject malformed zero-size tags and prove every pointer addition safe before dereference.

3. Start reserved, then release

`frame_bitmap` contains one bit per 4 KiB frame below 128 MiB. Initialization sets every bit to one: unavailable. Only page-aligned portions of available Multiboot ranges are cleared. This fail-closed order matters—unmentioned memory never silently becomes allocatable.

`make_available` rounds the start up and end down, clips the end to `MAX_PHYSICAL_MEMORY`, and clears full frames. The 128 MiB ceiling is a ToyOS teaching bound, not an x86-64 limit. It keeps the static bitmap small and matches QEMU's configured RAM, while deliberately ignoring higher RAM.

4. Reserve live boot objects again

After releasing usable ranges, `reserve_range` marks physical address zero through `__kernel_end` used, then reserves the Multiboot information block. This includes the image, bootstrap tables, static stacks, bitmap, and other linked storage because they lie below `__kernel_end`.

The second reservation is not redundant. Firmware can call a region usable while the bootloader has placed transient data inside it. The kernel owns that handoff until parsing is over; ToyOS conservatively retains it.

5. Audit allocator invariants in C

`frame_alloc` scans words and bits, sets one free bit, decrements `free_frames`, zeroes the frame, and returns its physical address. Zero is the failure sentinel, so frame zero is permanently reserved. `frame_free` checks alignment, bound, and nonzero before clearing.

Write these invariants beside the code:

```
bit 1 means unavailable or allocated
free_frames equals the number of zero bits in the bounded bitmap
an allocated frame is page-aligned, nonzero, and below 128 MiB
only the allocator changes frame ownership after initialization
```

`frame_free` now asks `frame_is_used` before clearing the bit. A second free therefore follows explicit misuse policy instead of silently leaving the count unchanged. There is still no owner record, reference count, lock, or per-CPU cache. Callers must transfer ownership exactly once.

6. Debug from the invariant outward

For an unexpected zero return, inspect `free_frames`, then the bitmap, then the parsed ranges and reservations. For corruption after allocation, verify that the identity map covers the returned physical frame before blaming the zeroing loop. In GDB, use the exact `build/week7/toyos.elf`; stale symbols can turn a correct address into a persuasive fiction.

The normal runtime line `SELFTEST: frame allocator PASS` proves one allocate/free round trip after initialization under this QEMU profile. The `expected-fault-week7-allocator` profile then frees that same frame again. `frame_free` observes a clear bitmap bit, emits `EXPECTED-FAULT: allocator PASS`, and exits through QEMU code `0x13`, which the host harness observes as status 39. Together these distinguish normal ownership transfer from one detected misuse. They do not prove exhaustion, fragmentation, concurrent calls, arbitrary firmware maps, or complete allocator safety.

Treat the injected case as controlled evidence:

```
allocate probe → bit becomes used → free probe → bit becomes free
→ inject second free → frame_is_used is false
→ expected allocator marker → checked host status 39
```

This is a double-free detector, not recovery. A non-test build panics on the same invalid transition.

7. Practical challenge

Use `challenges/day-001-memory-frames.sh` from the repository root.

Purpose: connect parser and bitmap invariants to the canonical Week 7 artifact.

Predict: state the allocator bound, page size, initial bit meaning, normal self-test marker, injected double-free marker, and host status 39 before running it.

Evidence: the wrapper shows relevant source contracts, checks the normal profile and `expected-fault-week7-allocator`, and inspects the exact `build/week7-allocator/toyos.elf`. Explain why the second call is rejected before `free_frames` can be incorrectly changed.

Mastery prompts

Explain why “available” is not yet “allocatable,” draw one unaligned memory range before and after page rounding, and reconstruct the exact bitmap state at both `frame_free(probe)` calls.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Memory Frames

Format: shell lab

What you will do: Audit the canonical Multiboot parser and bounded physical-frame allocator.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 07-memory-processes-and-scheduling/challenges/day-001-memory-frames.sh
```

- View lab source on GitHub

Day 2 — x86-64 Page Tables, the Mapper, TLBs, and Faults

Target time: approximately 3 hours

> A virtual address is a question asked of a hierarchy plus cached history. Editing a page-table entry changes memory; it does not automatically erase every translation the CPU may remember.

1. Walk the address, not the pointer expression

For the 48-bit canonical addresses used here, x86-64 divides a virtual address into PML4, PDPT, PD, PT, and 12-bit offset fields. Four table levels contain 512 eight-byte entries each. `CR3` identifies the root.

ToyOS's bootstrap in `projects/toyos/src/boot.S` uses 2 MiB identity mappings for the first GiB. The Week 7 mapper in `projects/toyos/src/memory.c` creates ordinary 4 KiB mappings outside that region. These facts coexist: page size is selected by entries in the walk, not globally.

2. Read `next_table` as ownership transfer

`next_table` tests `PTE_PRESENT`. If absent, it obtains a zeroed physical frame, installs its address with present/write flags, and thereby transfers that frame to page-table ownership. If an entry is huge, it returns `NULL` because this teaching mapper does not split a 2 MiB mapping.

The cast from entry address bits to `uint64_t *` works only because the bootstrap identity map reaches allocator results. In a higher-half kernel, the mapper would need a direct-map conversion or temporary mapping.

3. Map exactly one leaf

`map_page` extracts indices with shifts 39, 30, 21, and 12. It walks from `bootstrap_pml4`, refuses an already-present leaf, masks the physical address, allows only low flag bits, and forces present.

Audit assumptions the compact interface does not enforce:

- virtual and physical addresses should be page-aligned;
- the virtual address should be canonical;
- flags should represent an intentional supervisor policy;
- partially allocated intermediate tables are not rolled back on failure;
- there is one active address space and one CPU.

Those are ToyOS caller obligations. Production interfaces encode more of them and synchronize updates.

4. Invalidate cached translation state

After writing the PTE, ToyOS executes `invlpg` for the virtual address with a memory clobber. The compiler barrier and CPU invalidation solve different ordering problems. On one CPU with one address space, this is enough for the demonstration.

On SMP, every CPU that could have cached the old translation needs a TLB shootdown protocol. Changing `CR3`, PCID use, global mappings, and paging- structure caches complicate the exact rule. “We wrote the PTE” is not a complete mapping update story.

5. Diagnose a page fault as a tuple

The injected store in `projects/toyos/src/kernel.c` targets `0x70000000`. The handler in `projects/toyos/src/arch.c` combines:

```
CR2 fault address + page-fault error bits + saved RIP + exact ELF
```

The expected result is vector 14, address `0x70000000`, and `not-present write supervisor`. A protection fault would mean the translation exists but permissions reject the access. Reserved-bit and instruction-fetch bits answer different questions; do not collapse every #PF into “unmapped.”

6. Carry the model across architectures

AArch64 commonly walks translation tables rooted by `TTBR0_ELx` / `TTBR1_ELx`; fault diagnosis uses `ESR_ELx`, `FAR_ELx`, and `ELR_ELx`, and TLB maintenance uses explicit `TLBI` operations with ordering barriers.

RISC-V uses `satp` to select the address space and translation mode, page table entries appropriate to Sv39/Sv48, `sfence.vma` for translation-cache synchronization, and `scause`, `stval`, and `sepc` for diagnosis.

These are conceptual correspondences, not identical flags or ordering sequences. Consult the current architecture manual before writing low-level code.

7. Practical challenge

Use `challenges/day-002-mapper-fault.sh` from the repository root.

Purpose: tie a successful mapping and a deliberate missing mapping to specific source and dynamic evidence.

Predict: explain why `0x40000000` becomes writable while `0x70000000` faults, including the expected error classification.

Evidence: the wrapper checks the normal mapper marker and expected fault marker, then displays exact-ELF mapper symbols/disassembly and fault source. Static disassembly supports instruction identity; serial output supports the tested runtime path.

Mastery prompts

Draw all four indices for one ToyOS virtual address, state who owns each newly allocated table frame, and explain why a stale TLB entry can contradict the new bytes in memory without the CPU being “wrong.”

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Mapper Fault

Format: shell lab

What you will do: Contrast a canonical successful mapping with the injected missing mapping.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 07-memory-processes-and-scheduling/challenges/day-002-mapper-fault.sh
```

- View lab source on GitHub

Day 3 — Kernel Heap, Bump Allocation, and Ownership

Target time: approximately 3 hours

> Allocation answers “where?” Ownership answers “who may use it, for how long, and who must eventually release it?” ToyOS currently answers only part one.

1. Build the heap from lower layers

In `projects/toyos/src/memory.c`, `memory_init` obtains eight physical frames, maps them at `HEAP_BASE` (`0x50000000`), then initializes `heap_next` and `heap_end`. This order is important: returning a virtual pointer before backing it with page tables would make the allocator's first metadata access fault.

The heap is 32 KiB. It is virtual contiguous because the mapper installs consecutive virtual pages; its physical frames need not be contiguous.

2. Read the bump allocator carefully

`kalloc` rejects zero bytes, aligns the next pointer to 16 bytes, checks both the aligned start and remaining capacity, advances `heap_next`, and returns the old aligned start. The subtraction form `bytes > heap_end - aligned` avoids the overflow hazard of testing `aligned + bytes > heap_end`.

It does not clear each allocation, although newly allocated physical frames were zeroed earlier. It does not record size, type, owner, or call site. Alignment is sufficient for ToyOS task records and stacks, but it is policy, not a guarantee suitable for every future object.

3. Understand the attractive limitation

A bump allocator is tiny, deterministic, and difficult to corrupt because it has no free-list links. Allocation is constant-time. These are excellent bootstrap properties.

The price is permanent consumption. `kalloc` has no `kfree`, reuse, coalescing, guard pages, red zones, poisoning, or leak accounting. Repeated task creation would exhaust the heap even after tasks finish. A failed multi-step constructor also cannot roll back earlier allocations.

4. State ownership and lifetime in plain language

`create_task` in `projects/toyos/src/tasks.c` receives a task record and an 8192-byte stack. The scheduler's static `tasks[]` array retains pointers to both for the kernel lifetime. No code frees either. That is acceptable only because the demonstration creates exactly two permanent tasks.

For every pointer, write:

```
allocator: kalloc
owner: scheduler/task subsystem
lifetime: kernel lifetime in this milestone
aliases: tasks[id], current, local construction pointer
release: none; bump heap cannot reclaim
locking: none; single CPU, initialization before interrupts
```

This makes hidden assumptions reviewable.

5. C debugging hazards

Kernel allocators amplify ordinary C mistakes. An incorrect `size_t` calculation can under-allocate; pointer arithmetic can escape the object; misalignment can violate ABI or hardware requirements; a stale alias can outlive a future freed object. Compile-time warnings do not prove runtime bounds.

When debugging, inspect `heap_next`, `heap_end`, requested `bytes`, and the caller's expected object size. Use hexadecimal addresses and verify mappings. Do not print from allocator code after introducing locks unless the logging path is proven non-allocating and lock-safe.

6. Plan growth without pretending it exists

A next allocator might place headers in blocks, maintain size classes, or allocate whole pages for large objects. Then ToyOS would need double-free and use-after-free defenses, lock ownership, interrupt-context rules, and a clear relationship between virtual heap pages and physical-frame reclamation.

Linux uses multiple allocators and policies rather than one universal heap; source landmarks begin in `mm/`. xv6-riscv's `kernel/kalloc.c` is primarily a physical-page allocator, so it is not a direct replacement for this byte heap. Record source revisions before deeper claims.

7. Practical challenge

Use `challenges/day-003-kernel-heap.sh` from the repository root.

Purpose: audit heap construction, arithmetic, and current lifetime users.

Predict: compute capacity, alignment of successive small allocations, and why task allocations are never reclaimed.

Evidence: the wrapper shows heap constants and `kalloc`, checks the canonical heap self-test, and identifies all source call sites. The test is a single successful allocation, not an exhaustion or concurrency proof.

Mastery prompts

Explain why physical zeroing is not identical to allocation initialization, design an ownership record for one task stack, and name the first new invariant needed before adding `kfree`.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Kernel Heap

Format: shell lab

What you will do: Inspect heap construction, bump arithmetic, ownership, and current call sites.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 07-memory-processes-and-scheduling/challenges/day-003-kernel-heap.sh
```

- View lab source on GitHub

Day 4 — Kernel Tasks, Dedicated Stacks, and Saved Contexts

Target time: approximately 3 hours

> A task is not a function call. It is a resumable ownership bundle whose saved stack and registers let one instruction stream temporarily stop.

1. Name the abstraction accurately

`projects/toyos/src/tasks.c` defines two ring-0 kernel tasks. They share the same page tables, global variables, kernel code, devices, and privilege. They are not processes and do not provide isolation. Calling them processes would hide exactly what Week 8 must add.

Each `kernel_task` contains a saved `context`, state, identifier, entry function, and stack pointer. The static `tasks[2]` array is a tiny fixed run queue; `current` identifies the task whose stack and execution stream own the CPU.

2. Give every task a private kernel stack

`create_task` obtains an 8192-byte stack from the bump heap and computes an aligned top because x86-64 stacks grow downward. It subtracts eight bytes and places a zero sentinel so entry through a jump has a stack shape compatible with the function ABI's expectation that a call supplied a return address.

The initial saved `RSP` points there and initial `RIP` names `task_trampoline`. The context's other fields are zero from the heap's freshly zeroed physical pages, an assumption worth documenting rather than silently depending on forever.

No guard page catches stack overflow. An overflow can corrupt the adjacent heap object or scheduler metadata, producing symptoms far from the cause.

Before selecting a runnable task, `validate_context` now checks that its record and stack exist, its state is runnable, saved `RSP` lies inside the dedicated 8192-byte stack with the expected low bits, and saved `RIP` lies inside the linked kernel image. These are boundary checks, not proof that every byte of the saved continuation is trustworthy.

3. Distinguish two saved-state structures

`struct context` in `projects/toyos/include/toyos.h` stores the callee-saved registers, stack pointer, and resume address needed at a cooperative C call boundary. `struct interrupt_frame` stores a broader interrupted machine state assembled by hardware and `projects/toyos/src/isr.S`.

They are not interchangeable:

```
context: scheduler continuation at an ABI boundary
interrupt_frame: asynchronous exception/IRQ return state
```

A production preemptive scheduler usually coordinates these layers explicitly. ToyOS calls its compact context switch while already executing on the interrupted task's stack.

4. Enter through a trampoline

The switch jumps to `task_trampoline`, not directly to the task entry. The trampoline enables interrupts, verifies `current`, invokes the selected entry, and funnels a normal return into `task_finish`. This centralizes task exit policy.

`task_finish` marks the current task blocked, reports completion, and either schedules another task or exits after both finish and checks pass. “Blocked” here means permanently finished; there is no wait channel or wakeup path.

5. Track the state machine

The implemented transitions are:

```
created → RUNNABLE → RUNNING
RUNNING → RUNNABLE during a switch
RUNNING → BLOCKED on completion
```

There is no separate ready-list node, sleep queue, priority, fairness accounting, or idle task. If the other task is blocked, `schedule` may return without switching. State and queue membership are updated without locks. That relies on one CPU and carefully constrained interrupt behavior.

6. Debug task-start failures

If the first task never prints, inspect its saved `rip`, `rsp`, stack bounds, and alignment before investigating task logic. If a completed task resumes, check state selection and context ownership. If a return faults, inspect the sentinel and trampoline contract.

Use `build/week7/toyos.elf` with `gdb/week7.gdb`. A breakpoint at `context_switch` can inspect old context in `RDI` and new context in `RSI` under the System V AMD64 ABI.

The deterministic `expected-fault-week7-context` profile corrupts task 0's saved stack alignment by adding one to `context.rsp` after task creation. The first `schedule` calls `validate_context` before mutating `current` or jumping into assembly. The validator emits `EXPECTED-FAULT: context PASS` and QEMU code `0x14`; the host harness expects status 41. This establishes early rejection of that selected corruption, not safe handling of arbitrary stack or instruction-pointer damage.

7. Practical challenge

Use `challenges/day-004-task-context.sh` from the repository root.

Purpose: connect C layouts, initial stack construction, assembly offsets, and runtime task identity.

Predict: list context offsets and the first task's initial `RIP`; explain why its valid `RSP` ends in hexadecimal 8, why adding one violates the contract, and why rejection occurs before `context_switch`.

Evidence: the wrapper compiles an offset probe, compares switch assembly, checks both task-start markers, runs `expected-fault-week7-context`, and inspects its exact ELF. It does not establish stack-overflow safety, arbitrary-context recovery, or user-process isolation.

Mastery prompts

Draw both task stacks while task 1 runs, mark where task 0's resumable state lives, then trace the injected misalignment from `tasks_init` to `context_is_valid` and the checked host status.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Task Context

Format: shell lab

What you will do: Verify C context layout, switch offsets, stack construction, and task identity.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 07-memory-processes-and-scheduling/challenges/day-004-task-context.sh
```

- View lab source on GitHub

Day 5 — Cooperative Yield, Context Switch, and Register Ownership

Target time: approximately 3 hours

> A context switch does not move a running task. It changes which saved stack and continuation the CPU treats as current.

1. Build the CPU mental image

At any instant one CPU has one active `RSP` and instruction pointer. Other tasks are data structures containing enough state to become active later. ToyOS's `yielding_worker` voluntarily calls the scheduler four times; that C call is its yield operation even though there is no public function named `yield`.

Before a switch, identify:

```
CPU owner: current task
live state: registers plus its active stack
saved next task: context fields plus inactive stack
policy state: current, task states, fixed tasks[] order
```

2. Select before switching

`schedule` in `projects/toyos/src/tasks.c` picks task 0 initially, otherwise the next array slot. It avoids a blocked task and returns if no other runnable task exists. It then marks the old task runnable, selects an old-context destination, marks the new task running, updates `current`, and calls `context_switch`.

Updating `current` before assembly transfers control ensures the new task's trampoline and later code see the correct owner. It also means an interrupt at an unsafe point could observe partially transitioned scheduler state. ToyOS has no explicit interrupt-disable critical section around this path.

3. Let the ABI reduce saved state

`projects/toyos/src/switch.S` saves `R15`, `R14`, `R13`, `R12`, `RBX`, `RBP`, `RSP`, and a local resume `RIP`. These general registers are callee-saved by the System V AMD64 ABI. Caller-saved registers need not survive a normal C call to `context_switch`; the compiler already treats them as volatile.

The architecture does not define this set—the ABI does. SIMD/FPU state is absent, and kernel compilation uses `-mgeneral-regs-only`. Future code that uses extended state must add a policy for ownership and save/restore.

4. Follow the unusual return

Assembly stores the old `RSP` and the address of label `1`, loads the new callee-saved registers and `RSP`, then jumps to the new `RIP`.

For a new task, that target is `task_trampoline`. For a resumed task, it is label `1`, whose `ret` consumes the return address already present on that task's saved stack. Thus one invocation of `context_switch` may return much later, on the same task's stack, after several switches.

This is why stack ownership is central. A saved context whose stack storage was freed or overwritten is not a stale value—it is a future control-flow failure.

5. Separate mechanism from policy

Assembly implements mechanism: save old continuation, restore new continuation. C implements policy: which task should run and which states change. Keeping that boundary narrow makes it possible to audit offsets and ABI obligations.

The simple round-robin array is deterministic but not a general run queue. There is no priority, starvation protection, CPU affinity, or load balancing.

6. Diagnose context corruption

Typical signatures include:

- bad saved `RSP`: fault during restore or `ret`;
- bad saved `RIP`: jump into unmapped or non-code memory;
- offset mismatch: registers appear to rotate into one another;
- wrong `current`: one task mutates another task's state;
- stack overflow: scheduler data corrupts before the crash is observed.

In GDB, inspect both context pointers at entry, disassemble `context_switch`, and verify the destination addresses against `nm` from the exact ELF. C debugging still matters: enum state and pointer ownership bugs can perfectly feed assembly the wrong but well-aligned address.

7. Practical challenge

Use `challenges/day-005-cooperative-switch.sh` from the repository root.

Purpose: prove agreement among ABI expectations, C context order, assembly offsets, and the selected cooperative test.

Predict: identify every preserved register and narrate first-entry versus resume behavior.

Evidence: the wrapper inspects flags/layout/disassembly and checks the cooperative marker from the canonical profile. It cannot prove correctness under arbitrary interrupt timing.

Mastery prompts

Explain why `RAX` is not in `struct context`, trace the two different meanings of saved `RIP`, and identify the smallest region that would need interrupt exclusion before calling this scheduler robust.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Cooperative Switch

Format: shell lab

What you will do: Audit cooperative scheduling against the AMD64 ABI and exact switch code.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 07-memory-processes-and-scheduling/challenges/day-005-cooperative-switch.sh
```

- View lab source on GitHub

Day 6 — PIT/PIC Interrupt-Driven Preemption

Target time: approximately 3 hours

> Preemption means the current instruction stream did not volunteer. The interrupt path must preserve its claim on every register and stack byte before policy lends the CPU elsewhere.

1. Follow IRQ0 end to end

For milestone 7, `pic_and_pit_init` in `projects/toyos/src/arch.c` remaps the 8259 PIC so hardware IRQ0 arrives at IDT vector 32, unmasks only the timer, and programs PIT channel 0. `arch_init` installs `irq0_stub` at vector 32.

The path is:

```
PIT tick → PIC IRQ0 → IDT vector 32 → irq0_stub → isr_common
→ interrupt_frame on current stack → interrupt_handler
→ ticks++ → PIC EOI → scheduler_tick → schedule
```

The divisor 11932 against the traditional 1,193,182 Hz PIT input is roughly 100 Hz. Emulation and host scheduling mean this is not a precision timing measurement.

2. Account for interrupt context

`isr_common` in `projects/toyos/src/isr.S` saves all general-purpose registers represented by `interrupt_frame`, calls C on the interrupted task's current stack, then restores and executes `iretq`.

On the timer path, C may call `context_switch`, which changes stacks. The old task's stack retains its interrupt frame and nested C frames. When that task is scheduled again, `context_switch` returns through those frames, the handler returns, assembly restores the interrupt frame, and `iretq` resumes the interrupted instruction stream.

That compact design works only if every saved stack remains owned and intact.

3. Understand preemption evidence

`preempted_worker` never voluntarily schedules. It spins until the shared tick count advances by three. Task 1 can run only because timer interrupt handling invokes the scheduler. Therefore both the task output and `SELFTEST: PIT/PIC preemption PASS` are meaningful evidence for the selected path.

`volatile` on `task_done` prevents certain compiler caching, but it is not a lock or an inter-CPU memory-ordering protocol. `ticks` is changed in interrupt context and read in task context; this single-core build relies on aligned 64-bit operations and compiler-visible accesses.

4. Audit interrupt safety

Interrupt context cannot safely call arbitrary code. A production design must classify operations that may block, allocate, acquire contended locks, or recurse into interrupt-dependent services. ToyOS serial output polls, the heap has no lock, and the scheduler has no explicit critical section.

The handler sends PIC end-of-interrupt before scheduling. That releases the controller while CPU interrupts remain disabled from interrupt-gate entry until the task trampoline or eventual `iretq` enables/restores them. Nested timing and state updates deserve explicit design before adding devices or multiple CPUs.

5. Contrast modern interrupt controllers

The 8259 PIC and PIT are useful legacy teaching devices. Modern x86 systems normally use a local APIC timer, I/O APIC, MSI/MSI-X, and often x2APIC. Interrupt routing, per-CPU timers, acknowledgment, priorities, and SMP coordination differ.

Do not translate ToyOS port writes directly into a Linux explanation. Current Linux landmarks include `arch/x86/kernel/apic/`, `arch/x86/entry/`, and `kernel/sched/`, but names and paths require revision-specific inspection.

6. Locking assumptions are part of correctness

ToyOS assumes one CPU. No other core can concurrently mutate `current`, states, the frame bitmap, or heap cursor. Interrupts are the only asynchronous writer, and task initialization completes before they are enabled in the first trampoline.

Adding a second CPU would require per-CPU current-task state, run-queue locks or ownership partitioning, TLB shootdowns, interrupt routing, and memory ordering. “Add a spinlock” is not a sufficient architecture.

7. Practical challenge

Use `challenges/day-006-preemption.sh` from the repository root.

Purpose: connect PIT/PIC setup, IRQ frame construction, EOI, tick update, and an involuntary scheduling observation.

Predict: state the first vector, approximate frequency, register owner, and why task 1 can run while task 0 never yields.

Evidence: the wrapper displays the exact source chain and checks task, tick, and preemption markers in the canonical run. It does not validate APIC, SMP, latency, fairness, or physical timer accuracy.

Mastery prompts

Draw the interrupted task's stack at `scheduler_tick`, explain how it remains resumable across another task's execution, and list three operations that would need an interrupt-context safety contract before use in this handler.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Preemption

Format: shell lab

What you will do: Trace the canonical PIT/PIC IRQ0 path into involuntary task scheduling.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 07-memory-processes-and-scheduling/challenges/day-006-preemption.sh
```

- View lab source on GitHub

Day 7 — Scheduler Trace, Fault Diagnosis, and Systems Comparison

Target time: approximately 3 hours

> A scheduler report should make every ownership transfer reproducible and every uncertainty visible. “It printed PASS” is the beginning, not the end.

1. Trace state and CPU ownership

Build a timeline with one row per meaningful event:

```
event | task 0 state | task 1 state | current | active RSP owner | reason
```

Start after `tasks_init`: both tasks are runnable and `current` is null. `tasks_start` chooses task 0. Timer IRQ0 interrupts its busy loop, stores the interrupt continuation on task 0's stack, and switches to task 1. Task 1 voluntarily yields; later each task returns, becomes blocked, and the final task exits QEMU after both test conditions hold.

Do not imply a transition not represented in code. ToyOS has no real waiting queue: `TASK_BLOCKED` means finished.

2. Diagnose by violated invariant

The canonical project exposes three deterministic Week 7 profiles:

Allocator double-free (`EXPECT_FAULT=4`): `memory_init` allocates and frees `probe`, then injects a second `frame_free(probe)`. `frame_is_used` observes that its bitmap bit is already clear, so the handler emits `EXPECTED-FAULT: allocator PASS` and exits through QEMU code `0x13`. The host harness requires status 39. Inspect the exact `build/week7-allocator/toyos.elf`; do not describe this as allocator exhaustion, because that is not what was injected.

Unmapped page write (`EXPECT_FAULT=2`): the store to `0x70000000` produces vector 14 with CR2 equal to that address and `not-present write supervisor`. The handler exits through code `0x11`, and the host expects status 35. Correlate saved RIP with `build/week7-page/toyos.elf`.

Corrupted context (`EXPECT_FAULT=5`): `tasks_init` adds one to task 0's saved `rsp`. Before the first switch, `validate_context` rejects its lost 8-mod-16 alignment, emits `EXPECTED-FAULT: context PASS`, and exits through code `0x14`; the host expects status 41. Inspect `build/week7-context/toyos.elf`. Because rejection precedes `context_switch`, no invalid stack is loaded.

The three controls exercise different detection boundaries:

```
allocator metadata invariant → frame_free
address translation failure  → x86 #PF entry and handler
saved continuation invariant → scheduler validation
```

The wrappers invoke build profiles; they do not patch or restore canonical source.

3. Write the scheduler report

Include:

1. artifact identity and tool versions;
2. predictions for normal, allocator, page, and context profiles, including markers and host statuses 33, 39, 35, and 41;
3. exact source excerpts for allocation, mapping, task creation, switching, IRQ dispatch, and state transitions;
4. ELF hash and ordered symbols;
5. complete Week 7 test output;
6. the state/CPU/stack ownership trace;
7. interpretation of all three injected failures and next observations;
8. comparison and explicit limits.

Separate “source states,” “ELF contains,” “QEMU emitted,” and “we infer.”

4. Compare ToyOS with xv6-riscv

Useful current-tree landmarks commonly include:

- `kernel/kalloc.c`: physical-page allocation;
- `kernel/vm.c`: page-table construction and walking;
- `kernel/proc.c`: process state, allocation, and scheduling;
- `kernel/swtch.S`: saved kernel context;
- `kernel/trap.c`: timer/trap policy.

xv6 has processes, per-process page tables, user mode, syscalls, sleep/wakeup, and a richer lifecycle. Its RISC-V `swtch` and trap CSRs are conceptual counterparts, not x86 frame equivalents. Record the exact xv6 revision.

5. Compare ToyOS with Linux

Linux separates architecture entry and MMU code under `arch/x86/` from generic memory management in `mm/` and scheduler policy in `kernel/sched/`. It has per-CPU run queues, synchronization, priorities/classes, process and thread abstractions, user/kernel transitions, demand paging, and extensive fault recovery.

ToyOS's fixed two-element array can teach runnable/running ownership, but it does not model Linux fairness or SMP. Use current source navigation before naming functions; current paths are landmarks, not timeless contracts.

6. Preview processes and threads for Week 8

A process adds an isolation and resource-ownership boundary: user virtual address space, credentials/capabilities, files, signals, and lifecycle policy. A thread is an independently schedulable execution context that may share a process's address space and resources.

Week 7 kernel tasks already demonstrate stacks, saved execution contexts, and scheduling. Week 8 must add ring-3 entry, syscall and return ABI, user ELF loading, per-process page tables, validation of user pointers, and safe fault policy. Reusing the word “task” does not make those properties appear.

7. Practical challenge

Use `challenges/day-007-scheduler-report.sh` from the repository root.

Purpose: generate one temporary evidence report without editing canonical source or claiming unperformed live-debug observations.

Predict: write all four host statuses and exact markers, the page-fault tuple, the allocator bit transition, and the context alignment failure.

Evidence: the wrapper records source contracts, exact ELF identity, ordered symbols from every fault ELF, full Week 7 regression output covering all three injected failures, a static state trace, interpretations, comparisons, and limits. Add your causal explanation; generated text is not itself mastery.

Integrated mastery

Explain the complete chain from Multiboot available range to the final QEMU exit. At each boundary identify physical ownership, virtual mapping, C object lifetime, stack owner, register owner, interrupt state, and locking assumption. Then name the first Week 8 invariant that invalid user input will test.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Scheduler Report

Format: shell lab

What you will do: Generate a temporary Week 7 memory-and-scheduler evidence report.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 07-memory-processes-and-scheduling/challenges/day-007-scheduler-report.sh
```

- View lab source on GitHub